中山大學
SUN YAT-SEN UNIVERSITY



运算符重载（下）

中山大学计算机学院

主讲人：？？？

？？？@mail.sysu.edu.cn

目录

CONTENTS

01 下标运算符重载

02 函数对象

03 类外定义运算符重载

04 友元与友元函数

05 重载匹配与隐式转换

特殊运算符：下标

下标运算属于**成员访问运算符**。成员访问运算符的内建原型申明如下：

运算符名	语法	可重载	原型示例（对于 class T）	
			类定义内	类定义外
下标	a[b]	是	R& T::operator[](S b);	X 不适用
	a[...] (C++23 起)		R& T::operator[](...);	
间接寻址	*a	是	R& T::operator*();	R& operator*(T a);
取地址	&a	是	R* T::operator&();	R* operator&(T a);
对象的成员	a.b	否	不适用	不适用
指针的成员	a->b	是	R* T::operator->()	X 不适用
指向对象的成员的指针	a.*b	否	不适用	不适用
指向指针的成员的指针	a->*b	是	R& T::operator->*(S b);	R& operator->*(T a, S b);

注解

- 与大多数用户定义的重载相同，返回类型应当与内建运算符所提供的返回类型相匹配，以便用户定义的运算符可以和内建运算符以相同的方式使用。不过，在用户定义的运算符重载中，任何类型都可以作为它的返回类型（包括 void）。一个例外是 operator->，它必须返回一个指针或者另一个带有重载的 operator-> 的类以使它真正可用。

成员访问运算符 - cppreference.com

下标运算符：const与非const成员

C++规定，下标运算符[]必须以**成员函数的形式**进行重载。该重载函数在类中的声明格式如下：

```
R& T::operator[](S b);                    (1)

const R& T::operator[](S b) const;        (2)
```

(1)声明方式，[]不仅可以访问元素，还可以修改元素。

(2)声明方式，[]只能访问而不能修改元素。

在实际开发中，我们应该同时提供以上两种形式，这样做是为了适应 const 对象，因为通过**const对象只能调用 const成员函数**，如果不提供第二种形式，那么将无法访问 const 对象的任何元素。

下标运算符：示例

```
class IntArray  {
    int *a;
    int n;
public:
    IntArray(int n=1):n(n) {…}
    IntArray(const IntArray& other) {…}
    ~IntArray() {…}

    int& operator[](int i) {
        if (i>=0 && i < n) {
            return a[i];
        }
        throw std::out_of_range("out of range");
    }
    const int& operator[](int i) const{
        if (i>=0 && i < n) {
            return a[i];
        }
        throw std::out_of_range("out of range");
    }

    void print() const {…
    }
};
```

```
int main()  {
    IntArray a(4);
    for (int i=0;i<4;i++) a[i]=i+1;
    a.print();

    //任务1：以下两个语句，b的哪些成员函数被调用
    const IntArray b=a;
    cout << "b[0] = " << b[0] <<endl;

    //任务2：print函数去除 const会如何?
    b.print();

    //任务3:以下语句错误性质一样吗?
    //b[5]=10;
    //a[5]=10;

    return 0;
}
```

```
1 2 3 4
b[0] = 1
1 2 3 4
```

特殊运算符：函数调用

函数调用运算属于**其他运算符**。函数调用运算符的内建原型申明如下：

运算符名	语法	可重载	原型示例（对于 class T）	
			类内定义	类外定义
函数调用	a(a1, a2)	是	`R T::operator()(Arg1 &a1, Arg2 &a2, ...);`	X 不适用
逗号	a, b	是	`T2& T::operator,(T2 &b);`	`T2& operator,(const T &a, T2 &b);`
条件	a ? b : c	否	不适用	不适用

函数对象：函数调用 运算符为任何对象提供函数语义。当一个类 T 定义了函数调用，则T类型的对象 a 就可以类似函数一样使用 a(…)。 T 也称为**函数(callable)类**。

函数对象既有对象持**有状态**的特征，又有函数**可调用**特征。它在STL中传递函数类型参数、Lambda函数、闭包等现代语言元素实现等方面起核心作用。

其他运算符 - cppreference.com

函数对象 -无状态

当定义的 operator ()函数 没有使用对象的成员时，函数对象调用函数，其表现如同普通函数调用一般，例如：

```
class cmp {
  public:
    bool operator () (const int& a, const int& b)
{
        return a<b;
    }
};

int main()  {
    cmp f;
    cout<<f(1,2)<<endl;
    cout<<f(2,1)<<endl;
    return 0;
}
```

输出结果：
1
0

函数对象 - 有状态

当定义的 operator ()函数使用对象的成员时，函数对象调用函数，其形式如同普通函数调用一般，但其语义变为依赖对象本身内部状态。**同样的函数调用，结果却不同**。例如

```
class GreaterThan {
    int baseline;
public:
    GreaterThan(int x):baseline(x) { }
    bool operator () (const int& x) {
        return x>baseline;
    }
};

int main()  {
    GreaterThan g1(10), g2(20);
    cout << g1(15) << endl;
    cout << g2(15) << endl;
    return 0;
}
```

输出结果:

```
1
0
```

函数对象 vs 普通函数

- 谓词函数（Predicate）是布尔函数。对传进来的参数进行判断，并返回布尔值。
- 标准库中许多函数需要用户提供谓词函数作为参数，且支持任何**可调用**函数，即既支持普通函数作为实参，又支持函数对象，也支持 lambda 函数。比如：find_if，remove_if，等等。

现在假设我们有一串数字，要从中找出第一个大于10的数字以及第一个大于20的数字：

```
class GreaterThan {
    int baseline;
public:
    GreaterThan(int x):baseline(x) {}
    bool operator () (const int& x) {
        return x>baseline;
    }
};

bool GreaterThan10(const int& x)   {
    return x>10;
}
bool GreaterThan20(const int& x)   {
    return x>20;
}
```

```
int main()  {
    vector<int> a={5,10,15,20,25};
    // find_id return a iterator
    cout<<*find_if(a.begin(),a.end(),GreaterThan(10))<<endl;
    cout<<*find_if(a.begin(),a.end(),GreaterThan(20))<<endl;

    cout<<*find_if(a.begin(),a.end(),GreaterThan10)<<endl;
    cout<<*find_if(a.begin(),a.end(),GreaterThan20)<<endl;
    return 0;
}
```

类外定义运算符重载

问题：

- 用户自定义类的对象x也可以这样读入 `cin>>x` 和输出 `cout<<x` 吗？
- 第一操作数不是 `T` 类型左值对象，如 `Integer c = 4 + a;`

第一种情况：`cin` 的类定义时不一定知道 `x` 的类型存在，无法内部定义操作符重载。

第二种情况：第一操作数实参不是类型 `T`。

事实上，大多数运算符都可以类外定义！

类外定义运算符重载 - 案例

在类外定义一个普通的运算符重载函数。

```
struct Integer {
    int x;
public:
    Integer(int x=0):x(x) {}
    Integer operator- (const Integer& Int) {
        return Integer(x-Int.x);
    }
    Integer operator - () {
        return Integer(-x);
    }
    void print() {
        cout<<x<<endl;
    }
};

Integer operator+(const Integer& lhs,const Integer&
rhs) {
    return Integer(lhs.x+rhs.x);
}
```

```
int main()  {
    Integer a=3,b,c;

    //任务1：解释两个赋值语句，哪些函数被调用
    b = a + 4;
    c = 5 + a;
    b.print();
    c.print();

    //任务2：将 struct 关键字改为 class

    return 0;
}
```

```
7
8
```

友元函数 - 问题引入

运用类外定义，实现自己定义的类的读入、输出能重载是常见的事，但有时候类的成员是 private 的，外部函数无法访问，这又该如何解决?

```
class Integer {
    int x;
};

istream& operator >> (istream& is, Integer& Int)   {
    is>>Int.x; // error: private, thus unavailable
    return is;
}

ostream& operator << (ostream& os, const Integer& Int)
    os<<Int.x; // error: private, thus unavailable
    return os;
}
```

友元函数 - friend 关键字

友元函数（friend function）。

- `friend return_type function_name(parameter_type_list);`

顾名思义，友元函数就是给“朋友”看的函数，**类内部**声明 **friend 普通函数**，函数虽然**不属于类**，但却可以访问类的变量及函数（包括私有的）。**函数需要在类外实现**

```
class Integer {
    int x;
    friend istream& operator >> (istream& is, Integer& Int);
    friend ostream& operator << (ostream& os, const Integer& Int);
};

istream& operator >> (istream& is, Integer& Int)   {
    is>>Int.x; // private but friend, unavailable
    return is;
}

ostream& operator << (ostream& os, const Integer& Int)   {
    os<<Int.x; // private but friend thus unavailable
    return os;
}
```

友元函数 - 其他类成员函数作为友元

类的成员函数也是函数的一种，所以其他类的成员函数也可以是友元函数!

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    Integer(int x=0):x(x)   {
    }
    friend bool Cmp::operator () (const Integer& a, const Integer& b);
};

bool Cmp::operator () (const Integer& a, const Integer& b)   {
    return a.x<b.x;
}
```

友元类

但有些时候，其他类的成员函数可能会很多，一个一个的声明为友元函数会比较麻烦。

所以我们就可以直接声明**友元类**：

- 一个类 A 可以将另一个类 B 声明为自己的友元，那么类 B 的所有成员函数就都可以访问类 A 对象的私有成员
- `friend class B;` （在类 A 的内部）

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    Integer(int x=0):x(x)   {
    }
    friend Cmp;
};

bool Cmp::operator () (const Integer& a, const Integer& b)   {
    return a.x<b.x;
}
```

操作符重载 函数成员 or 友元函数?

必须是成员函数;

a=b	(a).operator= (b)	不能是非成员	给定 `std::string s;` , `s = "abc";` 调用 `s.operator=("abc")`
a(b...)	(a).operator() (b...)	不能是非成员	给定 `std::random_device r;` , `auto n = r();` 调用 `r.operator()()`
a[b]	(a).operator[](b)	不能是非成员	给定 `std::map<int, int> m;` , `m[1] = 2;` 调用 `m.operator[](1)`
a->	(a).operator-> ( )	不能是非成员	给定 `auto p = std::make_unique<S>();` `p->bar()` 调用 `p.operator->()`

问题引入

在学习了这节内容后，有同学可能会写出类似于下面的代码：

- Integer 可以利用字符串进行构造，并且也有一个名字为compare的函数对象
- 但是他之后又写了其他代码，写着写着他已经忘记了compare的定义...
- 而这时候他需要比较两个字符串的大小；哎，有个函数对象compare，他直接就调用了这个compare：

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    // atoi is a stl function that convert a c-string to int
    Integer(const char* s):x(atoi(s))   {
    }
    Integer(int x=0):x(x)   {
    }
    friend Cmp;
};

bool Cmp::operator () (const Integer& a, const Integer& b)   {
    return a.x<b.x;
}
```

```
int main()  {
    Integer a="3", b="4";
    Cmp compare;
    cout<<compare(a,b)<<endl;
    // some codes
    cout<<compare("a","b")<<endl; // unexpexcted behavior
    return 0;
}
```

对象隐式转换

如果对象 T 存在构造函数 T(T1)，则 T1 类型对象（或实参）可隐式转为 T 类型对象（或形参）

```
/* 隐式转换 */
...
class Integer {
    int x;
public:
    Integer(int x=0):x(x) {}
    friend Integer operator+(const Integer &lhs,const Integer &rhs){
        return lhs.x + rhs.x;
    }
    friend ostream& operator<<(ostream& o, const Integer &hs) {
        o << hs.x;
        return o;
    }
};
```

```
int main() {
    string s;
    s = "Hello";
    cout << s << endl;

    Integer i1(3), i2;
    i2 = 1.1 + i1;
    cout << i2 << endl;
}
```

请指出程序中得隐式转换?

对象隐式转换

如果对象 T 存在构造函数 T(T1)，则 T1 类型对象（实参）可隐式转为 T 类型对象（形参）

```
/* 隐式转换 */
...
class Integer {
    int x;
public:
    Integer(int x=0):x(x) {}
    friend Integer operator+(const Integer &lhs,const Integer &rhs){
        return lhs.x + rhs.x;   ① int -> Integer
    }
    friend ostream& operator<<(ostream& o, const Integer &hs) {
        o << hs.x;
        return o;
    }
};
```

```
int main() {
    string s;       ②
    s = "Hello";  const char* -> string
    cout << s << endl;

    Integer i1(3), i2;
    i2 = 1.1 + i1;          ③
    cout << i2 << endl;   double -> int
}                         int -> Integer
```

请指出程序中得隐式转换?

隐式转换-重载协议-const

如果重载的函数参数一样，可以通过转换到某个重载函数，编译会如何哪个版本的选择？

（1）类型直接匹配的优先选择；

（2）const 类型实参匹配 const 版本

（3）非 const 实参优先匹配非 const 版本。

没有则隐式转换为 const 版本匹配

程序运行结果：
no_const 1,3,const 2

观察与练习：
1. 注释去除非 const 版本，编译运行
2. 注释去除 const 版本，编译

```
/* 隐式转换-重载协议-const */
...
class Integer {
    int x;
public:
    Integer(int x=0):x(x) {}
    friend ostream& operator<<(ostream& o, const Integer &hs) {
        o << "const " << hs.x;
        return o;
    }
    friend ostream& operator<<(ostream& o, Integer &hs) {
        o << "no_const " << hs.x ;
        return o;
    }
};

int main() {
    Integer i1(1);
    const Integer i2(2);
    cout << i1 << "," << 3 << "," << i2 << endl;
}
```

隐式转换-nullptr

C 语言零值常数有很多表示，如 0，NULL，'\0'，C++ 右引入了 nullptr 表示空值指针字面量。

右边的例子解释了 nullptr 的必要性

```
/* 隐式转换-nullptr */
...
class Integer {
    int x;
public:
    Integer(int x=0):x(x) {}
    friend Integer operator+(const Integer &lhs, Integer rhs){
        return lhs.x + rhs.x;
    }
    friend Integer operator+(const Integer &lhs, Integer *rhs){
        if (rhs) return lhs.x + rhs->x + 2000;
        else return lhs.x + 1000;
    }
    friend ostream& operator<<(ostream& o, const Integer &hs) {
        o << hs.x; return o;
    }
};

class Girlfriend {/* ... */};
void kissGirlfriend(Girlfriend* gf) {cout << "pointer"<<endl;};
void kissGirlfriend(int gfID){cout << "int"<<endl;};

int main() {
    kissGirlfriend(nullptr);  // 指针类型字面量（viod *）0
    kissGirlfriend(0); // 尝试 NULL 取代 0

    Integer i1(1), i2;
    i2 = i1 + 0;  // 尝试 1,nullptr,NULL,&i1 取代 0
    cout << i2 << endl;
}
```

explicit

C++ 关键字 explicit，用于关闭这种自动类型转化的特性。

即被 explicit 关键字修饰的类构造函数，不能进行自动地隐式类型转换，只能显式地进行类型转换。

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    // atoi is a stl function that convert a c-string to int
    explicit Integer(const char* s):x(atoi(s))   {
    }
    Integer(int x=0):x(x)   {
    }
    friend Cmp;
};

bool Cmp::operator () (const Integer& a, const Integer& b)   {
    return a.x<b.x;
}
```

```
int main()  {
    Integer a("3"), b("4");
    Cmp compare;
    cout<<compare(a,b)<<endl; // of course ok
    // some codes
    cout<<compare(Integer("1"),Integer("2"))<<endl; // explicit construction, ok
    cout<<compare((Integer)"1",(Integer)"2")<<endl; // explicit conversion, ok
    cout<<compare("1","2")<<endl; // error
    cout<<compare("a","b")<<endl; // error
    return 0;
}
```

练习：Fraction

练习要求见 ex.cpp

中山大學
SUN YAT-SEN UNIVERSITY



谢 谢

中山大学计算机学院

主讲人：XXX

中山大学MOOC课程组